Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 7

з дисципліни «Основи клієнтської розробки»

Тема: «JavaScript. Події. Обробники подій. Спливання. Делегування подій.»

Виконав:
студент групи ІА-44
Соляник Денис.
Дата здачі 24.04.2026
Захищено з балом _____

Перевірила:
ст. вик. кафедри ІСТ
Хмелюк Марина Сергіївна

Київ 2026

Мета: навчитись використовувати події та писати обробники подій, застосовувати спливання та делегування подій в реалізації функціоналу.

Тема - Автомобілі

Завдання:

1) Використати одну з подій миші. Написати функцію-обробник. Призначити функцію-обробник події через атрибут і через властивість.

Використати метод addEventListener, призначити одній події різні обробники (написати функції-обробники).

Призначити обробником події об’єкт за допомогою addEventListener, застосувати метод handleEvent, вивести елемент, на якому спрацював обробник, використовуючи event.currentTarget

Видалити об’єкт, використовуючи removeEventListener

2) Створити список або використати існуючий. Реалізувати підсвічування елементів списку при кліку миші. Використати event.target. Обробник onclick застосувати для списку, а не для кожного елементу.

Створити меню (кілька кнопок), додати один обробник для всього меню і атрибути data-* для кожної кнопки, в відповідності з методами, які вони викликають.

Застосувати прийом проєктування «Поведінка» (додавання елементам поведінки behavior за допомогою атрибута data-*.

3) Завантажити проєкт на віддалений репозиторій (на GitHub).

Хід роботи:

Використано одну з подій миші. Написано функцію-обробник. Призначено функцію-обробник події через атрибут і через властивість.

Функція обробник

```
function activateEcoMode() {
  const originalBackground = document.body.style.backgroundColor;
  document.body.style.backgroundColor = "#d3d3d3";

  setTimeout(() => {
    document.body.style.backgroundColor = originalBackground;
    alert("Фон повернуто до попереднього кольору.");
  }, 30000);
```

```
}
```

Через атрибут

```
<div class="bottom-page-action">
            <button type="button" class="meta-button" id="background-btn"
onclick="activeEcoMode()">Тимчасово змінити фон</button>
        </div>
```

Через властивість

```
 const backgroundButton = document.getElementById('background-btn');
  if (backgroundButton) {
    backgroundButton.addEventListener('click', activateEcoMode);
  }
```

Використати метод addEventListener, призначити одній події різні обробники (написати функції-обробники).

```
function showTestDriveAlert() {
  updateEventStatus('Заявку на тест-драйв успішно прийнято менеджером
автосалону.', 'info');
}

function logTestDrive(){
  console.log('Кнопка була натиснута')
}
const showroomEventHandler = {
  handleEvent(event) {
    const activeElement = event.currentTarget;
    console.log('Обробник-об\'єкт спрацював на елементі:', activeElement);

    // const elementDescription =
`${activeElement.tagName.toLowerCase()}#${activeElement.id}`;
    // updateEventStatus(
    //   `Відділ тест-драйву зафіксував звернення клієнта. Канал прийому:
${elementDescription} (визначено через event.currentTarget).`,
    //   'success'
    // );

    activeElement.classList.add('is-active-handler');
    setTimeout(() => {
      activeElement.classList.remove('is-active-handler');
    }, 400);
  }
};
let isShowroomHandlerActive = false;

function attachShowroomEventHandler(testDriveBtn) {
```

```
  if (!testDriveBtn || isShowroomHandlerActive) {
    return;
  }

  testDriveBtn.addEventListener('click', showTestDriveAlert);
  testDriveBtn.addEventListener('click', logTestDrive);
  testDriveBtn.addEventListener('click', showroomEventHandler);
  testDriveBtn.disabled = false;
  isShowroomHandlerActive = true;
  updateEventStatus('Запис на тест-драйв ВІДКРИТО: менеджер фіксує звернення
клієнтів.', 'success');
}

function detachShowroomEventHandler(testDriveBtn) {
  if (!testDriveBtn || !isShowroomHandlerActive) {
    return;
  }

  testDriveBtn.removeEventListener('click', showTestDriveAlert);
  testDriveBtn.removeEventListener('click', logTestDrive);
  testDriveBtn.removeEventListener('click', showroomEventHandler);
  testDriveBtn.disabled = true;
  isShowroomHandlerActive = false;
  updateEventStatus('Запис на тест-драйв ЗАКРИТО: обробник заявок вимкнено',
'warning');
}
```

Призначити обробником події об’єкт за допомогою addEventListener, застосувати метод handleEvent, вивести елемент, на якому спрацював обробник, використовуючи event.currentTarget

```
const showroomEventHandler = {
  handleEvent(event) {
    const activeElement = event.currentTarget;
    console.log('Обробник-об\'єкт спрацював на елементі:', activeElement);

    // const elementDescription =
`${activeElement.tagName.toLowerCase()}#${activeElement.id}`;
    // updateEventStatus(
    //   `Відділ тест-драйву зафіксував звернення клієнта. Канал прийому:
${elementDescription} (визначено через event.currentTarget).`,
    //   'success'
    // );

    activeElement.classList.add('is-active-handler');
    setTimeout(() => {
      activeElement.classList.remove('is-active-handler');
    }, 400);
  }
```

```
};
```

Видалити об’єкт, використовуючи removeEventListener

```
function detachShowroomEventHandler(testDriveBtn) {
  if (!testDriveBtn || !isShowroomHandlerActive) {
    return;
  }

  testDriveBtn.removeEventListener('click', showTestDriveAlert);
  testDriveBtn.removeEventListener('click', logTestDrive);
  testDriveBtn.removeEventListener('click', showroomEventHandler);
  testDriveBtn.disabled = true;
  isShowroomHandlerActive = false;
  updateEventStatus('Запис на тест-драйв ЗАКРИТО: обробник заявок вимкнено',
'warning');
}
```

Створити список або використати існуючий. Реалізувати підсвічування елементів списку при кліку миші. Використати event.target. Обробник onclick застосувати для списку, а не для кожного елементу.

```
<ul id="inventory-list" class="inventory-list">
        <li class="inventory-item" data-model="BMW 320i">
            <span class="inventory-brand">BMW 320i</span>
            <span class="inventory-note">Спортивний характер та інновації.</span>
        </li>
        <li class="inventory-item" data-model="Audi A6">
            <span class="inventory-brand">Audi A6</span>
            <span class="inventory-note">Технологічність та повний привід
quattro.</span>
        </li>
        <li class="inventory-item" data-model="Mercedes C-Class">
            <span class="inventory-brand">Mercedes C-Class</span>
            <span class="inventory-note">Комфорт класу люкс та статус.</span>
        </li>
        <li class="inventory-item" data-model="Lexus RX">
            <span class="inventory-brand">Lexus RX</span>
            <span class="inventory-note">Надійність, безпека і преміальна
якість.</span>
        </li>
    </ul>
```

```
 const inventoryList = document.getElementById('inventory-list');
  if (inventoryList) {
    inventoryList.onclick = (event) => {
      const clickedItem = event.target.closest('.inventory-item');
```

```
      if (!clickedItem || !inventoryList.contains(clickedItem)) {
        return;
      }

      const activeItem = inventoryList.querySelector('.inventory-item--active');
      if (activeItem) {
        activeItem.classList.remove('inventory-item--active');
      }

      clickedItem.classList.add('inventory-item--active');
      updateEventStatus(
        `Обрано модель: ${clickedItem.dataset.model}. Менеджер готує деталі
комплектації.`,
        'info'
      );
    };
  }
```

Створити меню (кілька кнопок), додати один обробник для всього меню і атрибути data-* для кожної кнопки, в відповідності з методами, які вони викликають.

Застосувати прийом проєктування «Поведінка» (додавання елементам поведінки behavior за допомогою атрибута data-*.

```
<div class="action-grid" id="catalog-menu">
            <button type="button" class="action-button action-button--navy"
id="consultation-btn" data-action="requestConsultation">
                <span class="btn-title">Замовити консультацію</span>
                <span class="btn-subtitle">Швидко залишити заявку</span>
            </button>
            <button type="button" class="action-button action-button--sky"
id="offers-btn" data-action="showOffers">
                <span class="btn-title">Поточні акції</span>
                <span class="btn-subtitle">Переглянути вигідні умови</span>
            </button>
            <button type="button" class="action-button action-button--brick"
id="home-btn" data-action="goHome">
                <span class="btn-title">На головну</span>
                <span class="btn-subtitle">Повернутися до автосалону</span>
            </button>
            <button type="button" class="action-button action-button--green"
id="drive-btn" data-action="registerTestDrive">
                <span class="btn-title">Тест-драйв</span>
                <span class="btn-subtitle">Записатися на тест-драйв</span>
            </button>
        </div>
```

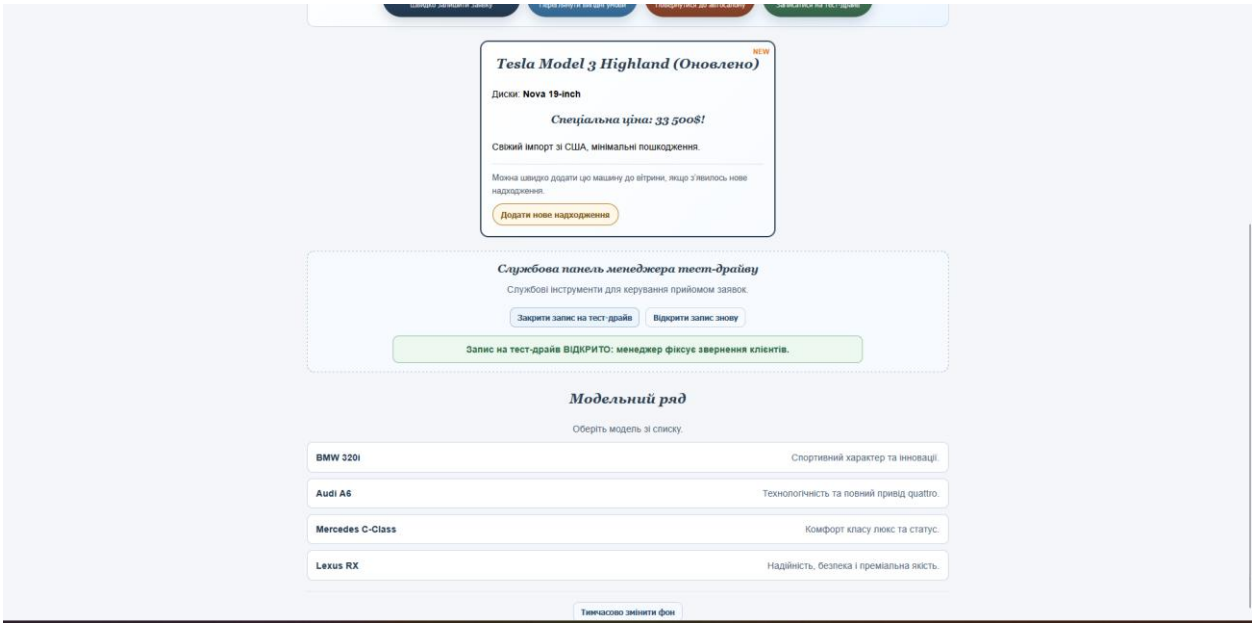
```
const catalogMenu = document.getElementById('catalog-menu');
  if (catalogMenu) {
```

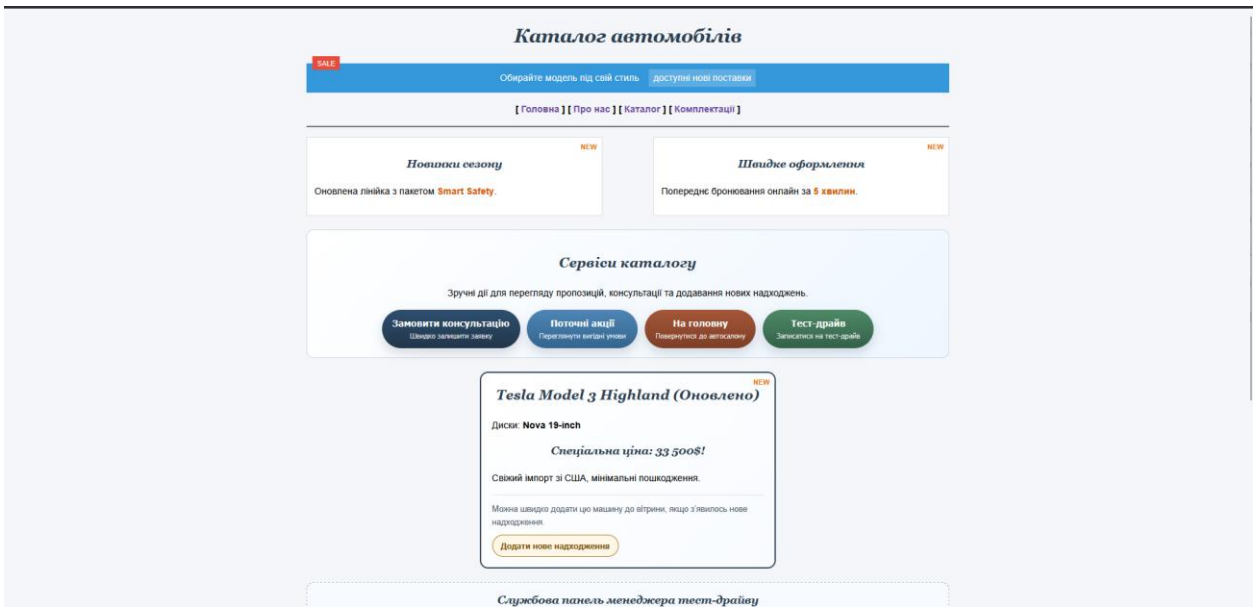
```
    const menuBehavior = {
      requestConsultation() {
        requestConsultation();
      },
      showOffers() {
        showCurrentOffers();
      },
      goHome() {
        location.href = 'index.html';
      },
      registerTestDrive() {
        updateEventStatus('Клієнт натиснув кнопку запису на тест-драйв у
каталозі.', 'info');
      }
    };

    catalogMenu.onclick = (event) => {
      const actionButton = event.target.closest('[data-action]');
      if (!actionButton || !catalogMenu.contains(actionButton)) {
        return;
      }

      const actionName = actionButton.dataset.action;
      const actionMethod = menuBehavior[actionName];
      if (typeof actionMethod === 'function') {
        actionMethod();
      }
    };
  }
```

Результат:





Висновок:

Під час виконання лабораторної роботи на базі вебсторінки автомобільної тематики було успішно закріплено механізми роботи з подіями в JavaScript: відпрацьовано призначення обробників через HTML-атрибути, властивості DOM та addEventListener, а також використання об'єктів як обробників через handleEvent із їх динамічним видаленням. Логіку інтерактивних елементів було значно оптимізовано завдяки механізму спливання та делегуванню подій на батьківські контейнери за допомогою event.target. Крім того, успішне впровадження патерну «Поведінка» (Behavior) із використанням data-* атрибутів дозволило реалізувати гнучке керування меню через єдиний обробник, що в сукупності сприяло написанню чистого, масштабованого коду та закріпленню фундаментальних навичок Frontend-розробки.